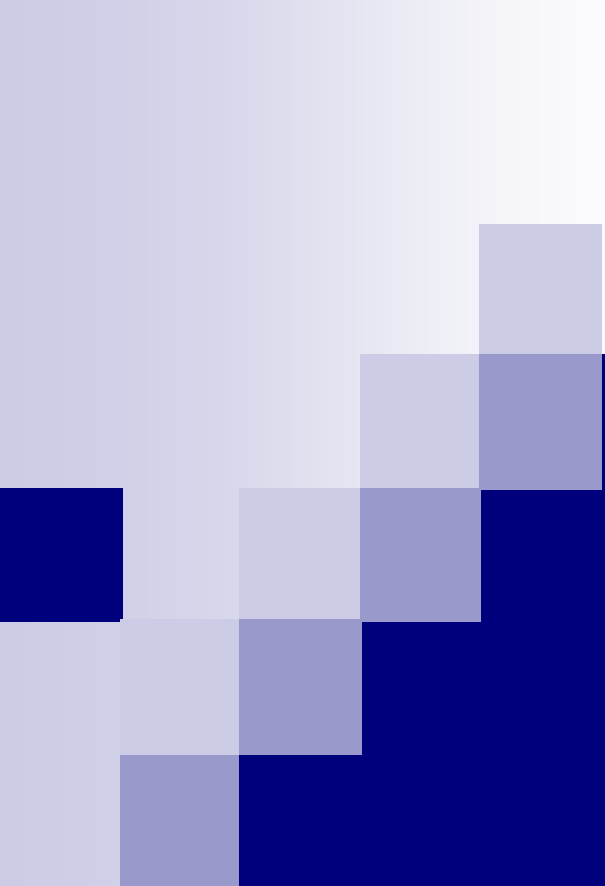




Computação Eletrônica

Variáveis Ponto
Flutuante e
Booleanas

Expressões inteiras (revisão)

- Operações:

- ☐ Soma +
- ☐ Subtração –
- ☐ Multiplicação *
- ☐ Divisão /

- $11 / 4 = 2$

- $11 \% 4 = 3$

- $x*10+3$

- $10*3 / 4 = (10*3) / 4 = 7$

- Precedência: * / % + -

Exercício

- Programa C que recebe 3 notas inteiras e calcula a média

```
#include<stdio.h>
int main () {
    int nota1, nota2, nota3, media;
    printf("Entre com a primeira nota: ");
    scanf("%d",&nota1);
    printf("Entre com a segunda nota: ");
    scanf("%d", &nota2);
    printf("Entre com a terceira nota: ");
    scanf("%d", &nota3);
    media = (nota1+nota2+nota3) / 3;
    printf("A media foi: %d", media);
    return 0;
}
```

Veja que o programa sempre calcula a média arredondada para um número inteiro.

Case Sensitive

- A linguagem C faz diferenciação de letras maiúsculas e minúsculas

```
#include<stdio.h>
int main() {
    int y = 3;
    printf("%d",Y); // Erro de compilação"
    return 0;
}
```

Constantes

- Não é possível modificar o valor

```
#include<stdio.h>

int main() {
    const int x = 3;
    printf("%d", x);
    return 0;
}
```

Constantes

- Não é possível modificar o valor

```
#include <stdio.h>

int main() {
    const int x = 3;
    x = 4; // "Erro de compilação"
    printf("%d", x);
    return 0;
}
```

Variáveis de Ponto Flutuante (Números Reais)

- float: 32 bits; $\pm 3.4 \cdot 10^{-38}$ a $3.4 \cdot 10^{38}$
- double: 64 bits; $\pm 1.7 \cdot 10^{-308}$ a $1.7 \cdot 10^{308}$
- long double: 80 bits; $\pm 3.4 \cdot 10^{-4932}$ a $3.4 \cdot 10^{4932}$
- Exemplo

```
int main() {  
    float x;  
    return 0;  
}
```

Expressões Reais

- Operações:

- ☐ Soma +
- ☐ Subtração –
- ☐ Multiplicação *
- ☐ Divisão /

- $11/4 = 2.750000$

- $x*10.5+3$

- $10.5*3 / 4 = (10.5*3) / 4 = 7.87500$

printf e scanf

Tipo	printf	scanf
float	%f	%f
double	%f ou %lf	%lf
long double	%Lf	%Lf

Obs: No printf, %lf tem o mesmo efeito de %f para double, e %f pode ser usado diretamente. Para facilitar a memorização e uso, adote %lf no printf e scanf para double.

printf

```
#include<stdio.h>
```

```
int main() {  
    float x = 2.78;  
    printf("%f \n",x);  
    printf("%.1f \n",x);  
    printf("%10f \n",x);  
    printf("%10.1f",x);  
    return 0;  
}
```

" 2.780000"

"2.8"

" 2.780000"

" 2.8"

1 casa decimal

Número de espaços reservado para imprimir o número.

printf

O que vai sair no monitor?

```
#include<stdio.h>
int main() {
    float x = 2.78;
    int y = 3;
    int z = 4;
    printf("Os tres numeros sao %.1f %d %d", x, y, z);

    return 0;
}
```

scanf

```
#include<stdio.h>
```

*O usuário pode entrar
com 3 números
separados por espaço*

```
int main() {
```

```
    float x, y, z;
```

```
    printf("Digite 3 numeros: ");
```

```
    scanf("%f %f %f", &x, &y, &z);
```

```
    printf("Voce digitou %f %f %f", x, y, z);
```

```
    return 0;
```

```
}
```

Funções de Ponto Flutuante

Colocar no início do programa `#include<math.h>`

Função	Descrição
log(x)	Logaritmo natural de x
exp(x)	e elevado a x
abs(x)	Valor absoluto
trunc(x)	Trunca x
round(x)	Arredonda x
pow(x,y)	x elevado a y
sqrt(x)	Raiz quadrada de x
sin(x)	Seno de x
cos(x)	Cosseno de x
atan(x)	Arco tangente de x

Variáveis de Ponto Flutuante

```
#include<stdio.h>
int main () {
    float nota1, nota2, nota3, media;
    printf("Entre com a primeira nota: ");
    scanf("%f",&nota1);
    printf("Entre com a segunda nota: ");
    scanf("%f", &nota2);
    printf("Entre com a terceira nota: ");
    scanf("%f", &nota3);
    media = (nota1+nota2+nota3) / 3;
    printf("A media foi: %.2f", media);
    return 0;
}
```

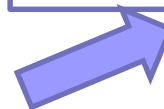
Variáveis de Ponto Flutuante

```
#include<stdio.h>
int main () {
    int nota1, nota2, nota3; float media;
    printf("Entre com a primeira nota: ");
    scanf("%d",&nota1);
    printf("Entre com a segunda nota: ");
    scanf("%d", &nota2);
    printf("Entre com a terceira nota: ");
    scanf("%d", &nota3);
    media = (nota1+nota2+nota3) / 3;
    printf("A media foi: %.2lf", media);
    return 0;
}
```

Variáveis de Ponto Flutuante

```
#include<stdio.h>
int main () {
    int nota1, nota2, nota3; float media;
    printf("Entre com a primeira nota\n");
    scanf("%d",&nota1);
    printf("Entre com a segunda nota\n");
    scanf("%d", &nota2);
    printf("Entre com a terceira nota\n");
    scanf("%d", &nota3);
    media = (nota1+nota2+nota3) / 3;
    printf("A media foi: %.2lf", media);
    return 0;
}
```

Todos operandos são inteiros. Cuidado, divisão de números inteiros! Necessário usar operador de cast (conversão de tipos)



Variáveis de Ponto Flutuante

```
#include<stdio.h>

int main () {
    int nota1, nota2, nota3; float media;
    printf("Entre com a primeira nota: ");
    scanf("%d",&nota1);
    printf("Entre com a segunda nota: ");
    scanf("%d", &nota2);
    printf("Entre com a terceira nota: ");
    scanf("%d", &nota3);
    media = (float) (nota1+nota2+nota3) / 3;
    printf("A media foi: %.2lf", media);
    return 0;
}
```

*A expressão
(float) (nota1+nota2+nota3)
converte (nota1+nota2+nota3), que
é um int, para float.*

*O ideal é não ter que
fazer conversões, mas
já declarar as
variáveis no tipo
certo.*

Expressões Booleanas (Lógicas)

- Podem ser Verdadeiro ou Falso
- C não possui um tipo explícito para lógica
- Uso de valores numéricos
- Falso é representado por 0
- Verdadeiro é representado por qualquer valor diferente de zero.
 - 1 é comumente utilizado para representar Verdadeiro
 - 1 é adotado pela linguagem C após a avaliação de uma expressão

Operadores Relacionais

- $3 == 4$ (igualdade)
- $3 != 4$ (diferente)
- $3 \leq 4$
- $3 < 4$
- $3 > 4$
- $3 \geq 4$
- Também podemos usar variáveis inteiras ou reais em expressões booleanas:
 - $X \geq 4$
 - $Y < 3.5$

Operadores relacionais

```
#include<stdio.h>

int main() {
    printf("3 == 4 eh: %d\n", 3 == 4);
    printf("3 != 4 eh: %d\n", 3 != 4);
    printf("3 <= 4 eh: %d\n", 3 <= 4);
    printf("3 < 4 eh: %d\n", 3 < 4);

    return 0;
}
```

Execução

```
#include<stdio.h>
```

```
int main() {
```

```
    printf("3 == 4 eh: %d\n", 3 == 4);
```

```
    printf("3 != 4 eh: %d\n", 3 != 4);
```

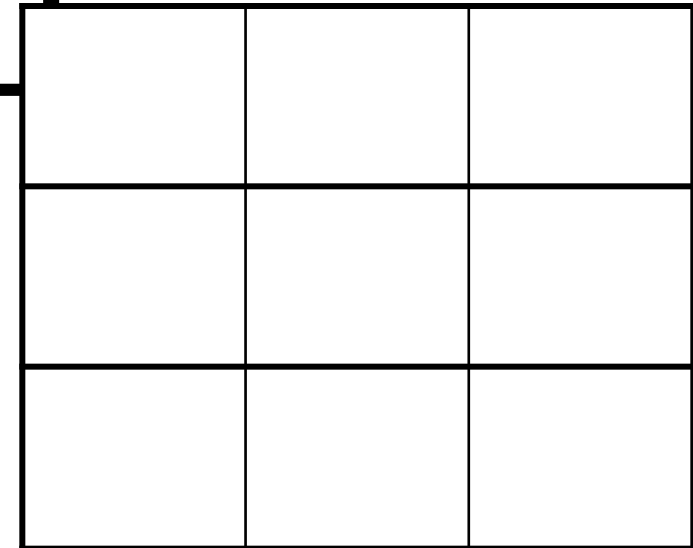
```
    printf("3 <= 4 eh: %d\n", 3 <= 4);
```

```
    printf("3 < 4 eh: %d\n", 3 < 4);
```

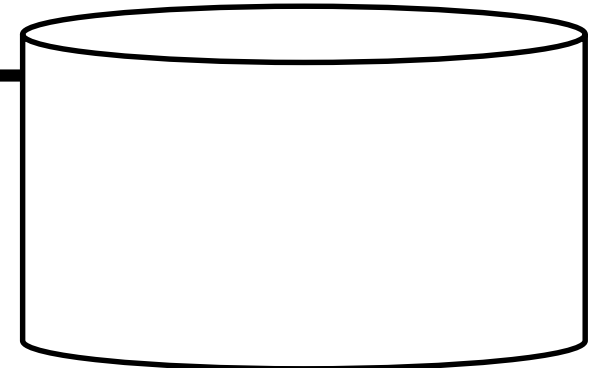
```
    return 0;
```

```
}
```

Memória



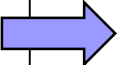
HD



Execução

```
#include<stdio.h>
```

```
int main() {
```



```
printf("3 == 4 eh: %d\n", 3 == 4);
```

```
printf("3 != 4 eh: %d\n", 3 != 4);
```

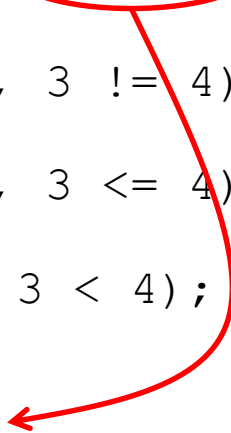
```
printf("3 <= 4 eh: %d\n", 3 <= 4);
```

```
printf("3 < 4 eh: %d\n", 3 < 4);
```

```
return 0;
```

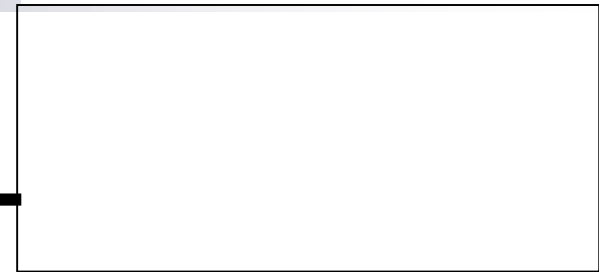
```
}
```

3 == 4?
= 0 (falso)



Antes de fazer o printf(), o computador avalia se 3 é igual a 4.

Se for, ele retorna 1. Senão, ele retorna 0. Neste caso, retorna-se 0.



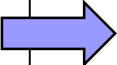
Memória

HD

Execução

```
#include<stdio.h>
```

```
int main() {
```



```
printf("3 == 4 eh: %d\n", 3 == 4);
```

```
printf("3 != 4 eh: %d\n", 3 != 4);
```

```
printf("3 <= 4 eh: %d\n", 3 <= 4);
```

```
printf("3 < 4 eh: %d\n", 3 < 4);
```

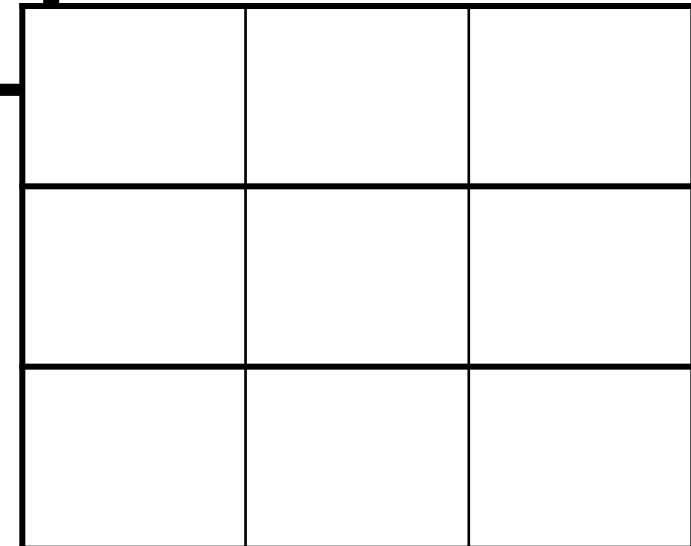
```
return 0;
```

```
}
```

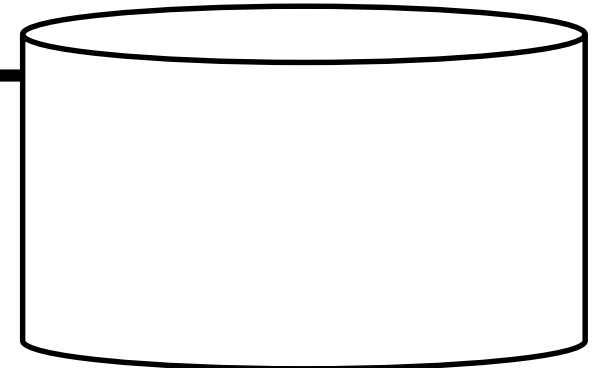
*3 == 4?
= 0 (falso)*

3 == 4 eh: 0

Memória



HD

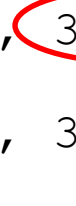


Execução

```
#include<stdio.h>
```

```
int main() {
```

```
    printf("3 == 4 eh: %d\n", 3 == 4);
```



```
    printf("3 != 4 eh: %d\n", 3 != 4);
```

```
    printf("3 <= 4 eh: %d\n", 3 <= 4);
```

```
    printf("3 < 4 eh: %d\n", 3 < 4);
```

```
    return 0;
```

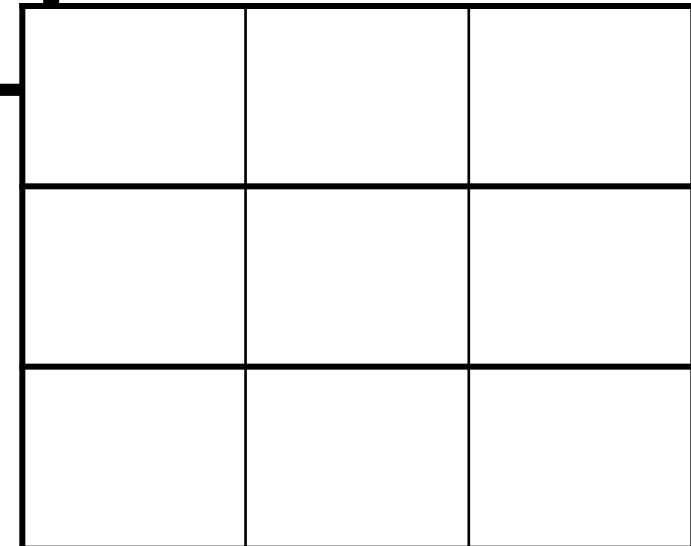
*3 != 4?
=1 (verdadeiro)*

Antes de fazer o printf(), o computador avalia se 3 é diferente de 4.

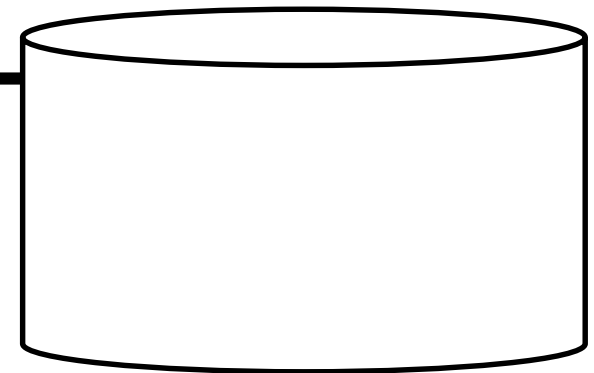
Se for, ele retorna 1. Senão, ele retorna 0. Neste caso, retorna-se 1.

3 == 4 eh: 0

Memória



HD

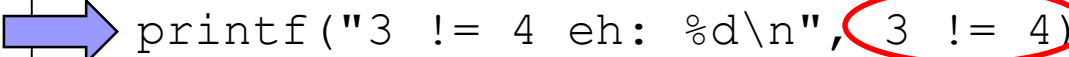


Execução

```
#include<stdio.h>
```

```
int main() {
```

```
    printf("3 == 4 eh: %d\n", 3 == 4);
```



```
    printf("3 != 4 eh: %d\n", 3 != 4);
```

```
    printf("3 <= 4 eh: %d\n", 3 <= 4);
```

```
    printf("3 < 4 eh: %d\n", 3 < 4);
```

```
    return 0;
```

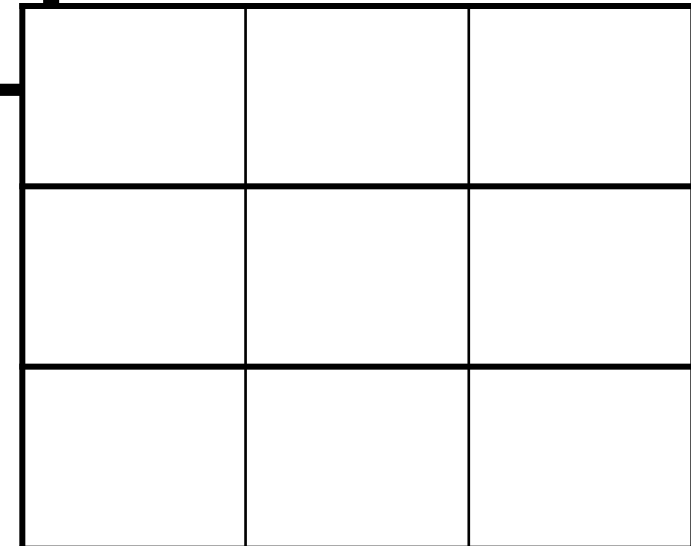
```
}
```

*3 != 4?
=1 (verdadeiro)*

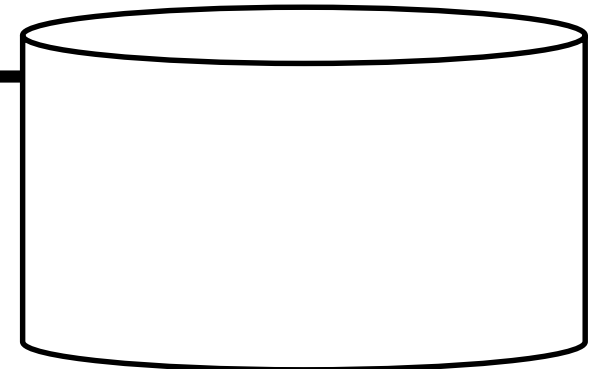
3 == 4 eh: 0

3 != 4 eh: 1

Memória



HD



Execução

```
#include<stdio.h>

int main() {

    printf("3 == 4 eh: %d\n", 3 == 4);

    printf("3 != 4 eh: %d\n", 3 != 4);

    printf("3 <= 4 eh: %d\n", 3 <= 4);

    printf("3 < 4 eh: %d\n", 3 < 4);

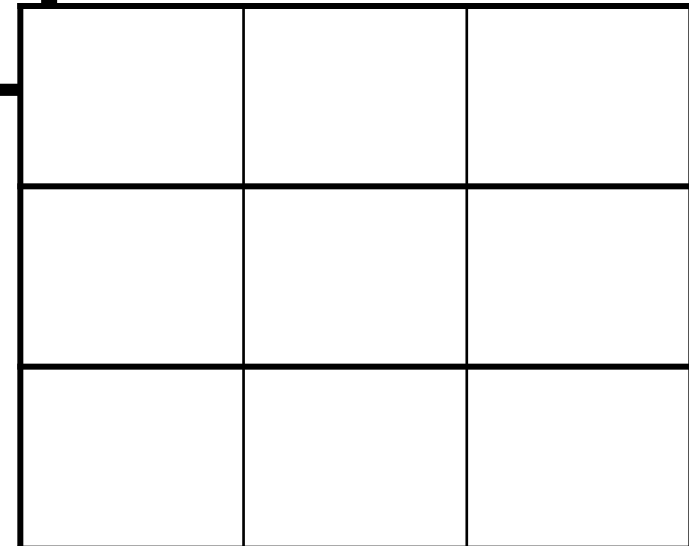
    return 0;
}
```

*3 <= 4?
=1 (verdadeiro)*

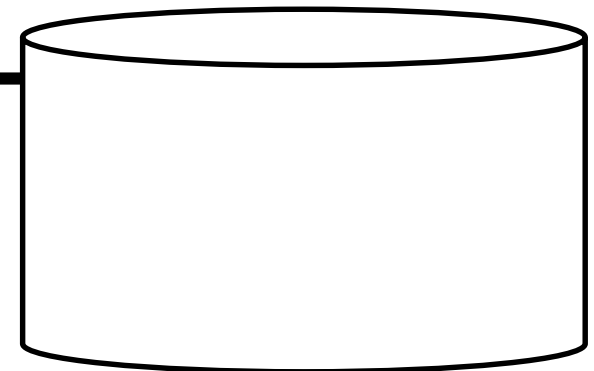
Antes de fazer o printf(), o computador avalia se 3 é menor ou igual a 4. Se for, ele retorna 1. Senão, ele retorna 0. Neste caso, retorna-se 1.

3 == 4 eh: 0
3 != 4 eh: 1

Memória



HD



Execução

```
#include<stdio.h>
```

```
int main() {
```

```
    printf("3 == 4 eh: %d\n", 3 == 4);
```

```
    printf("3 != 4 eh: %d\n", 3 != 4);
```

```
    printf("3 <= 4 eh: %d\n", 3 <= 4);
```

```
    printf("3 < 4 eh: %d\n", 3 < 4);
```

```
    return 0;
```

```
}
```

*3 <= 4?
=1 (verdadeiro)*

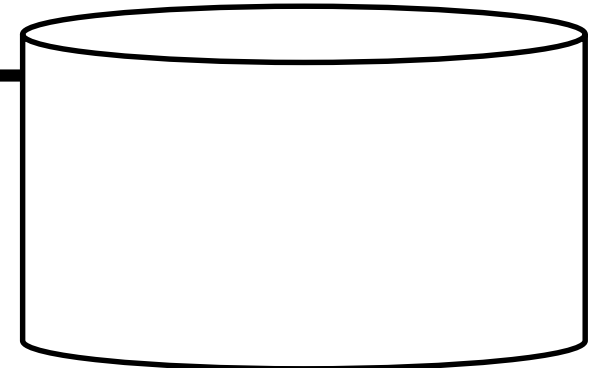
3 == 4 eh: 0

3 != 4 eh: 1

3 <= 4 eh: 1

Memória

HD



Execução

```
#include<stdio.h>
```

```
int main() {
```

```
    printf("3 == 4 eh: %d\n", 3 == 4);
```

```
    printf("3 != 4 eh: %d\n", 3 != 4);
```

```
    printf("3 <= 4 eh: %d\n", 3 <= 4);
```

```
    printf("3 < 4 eh: %d\n", 3 < 4);
```

```
    return 0;
```

```
}
```

*3 < 4?
=1 (verdadeiro)*

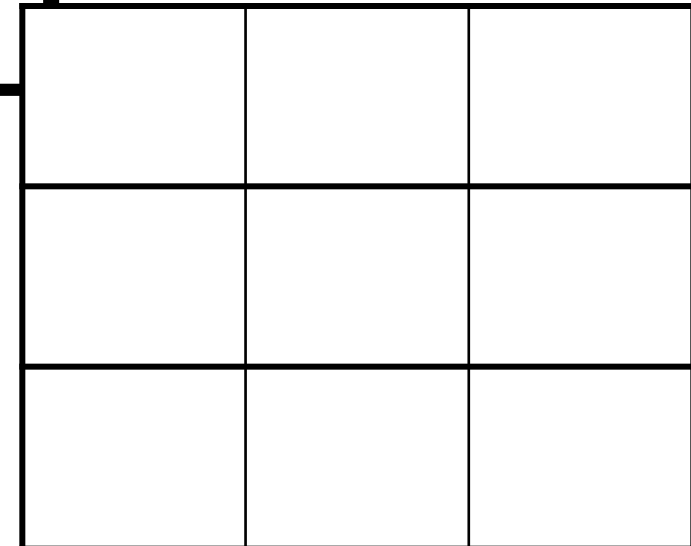
Antes de fazer o printf(), o computador avalia se 3 é menor que 4. Se for, ele retorna 1. Senão, ele retorna 0. Neste caso, retorna-se 1.

3 == 4 eh: 0

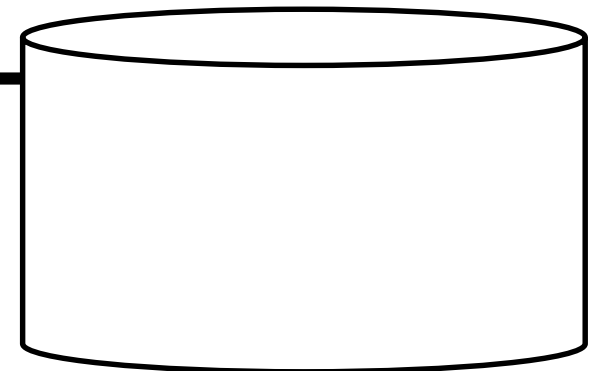
3 != 4 eh: 1

3 <= 4 eh: 1

Memória



HD



Execução

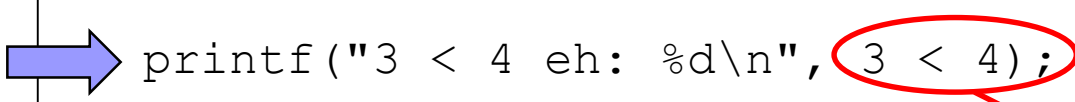
```
#include<stdio.h>
```

```
int main() {
```

```
    printf("3 == 4 eh: %d\n", 3 == 4);
```

```
    printf("3 != 4 eh: %d\n", 3 != 4);
```

```
    printf("3 <= 4 eh: %d\n", 3 <= 4);
```



```
    printf("3 < 4 eh: %d\n", 3 < 4);
```

```
    return 0;
```

```
}
```

*3 < 4?
=1 (verdadeiro)*



3 == 4 eh: 0

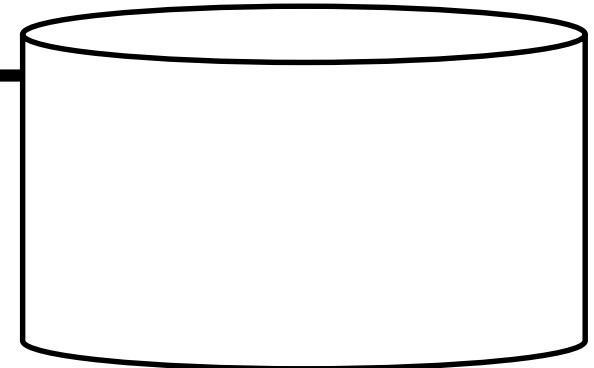
3 != 4 eh: 1

3 <= 4 eh: 1

3 < 4 eh: 1

Memória

HD



Execução

```
#include<stdio.h>

int main() {

    printf("3 == 4 eh: %d\n", 3 == 4);

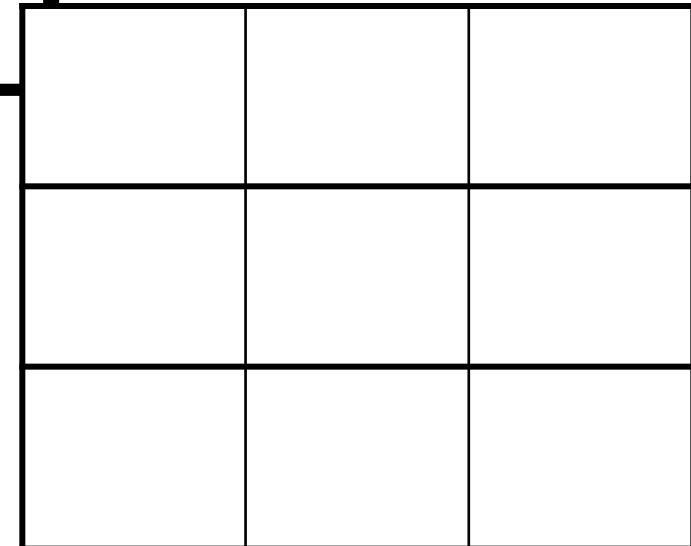
    printf("3 != 4 eh: %d\n", 3 != 4);

    printf("3 <= 4 eh: %d\n", 3 <= 4);

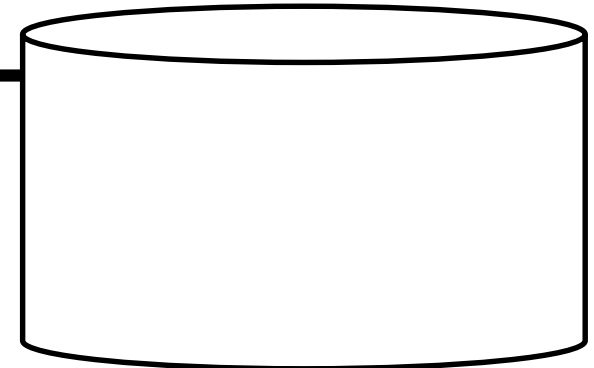
    printf("3 < 4 eh: %d\n", 3 < 4);

    return 0;
}
```

Memória



HD



Operadores relacionais

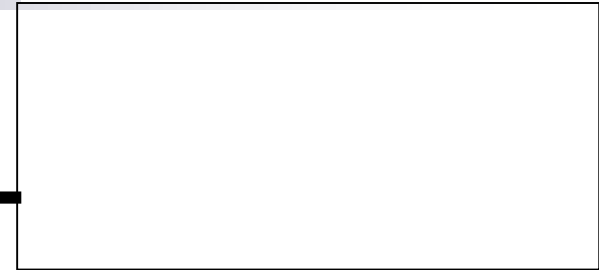
```
#include<stdio.h>
int main() {
    float x;
    int resposta;
    printf("Digite um numero e direi se eh maior que 20: ");
    scanf("%f", &x);
    resposta = x > 20;
    printf("Resposta: %d\n", resposta);
    printf("Legenda: 0 significa NAO e 1 significa SIM.\n");
    return 0;
}
```

Execução

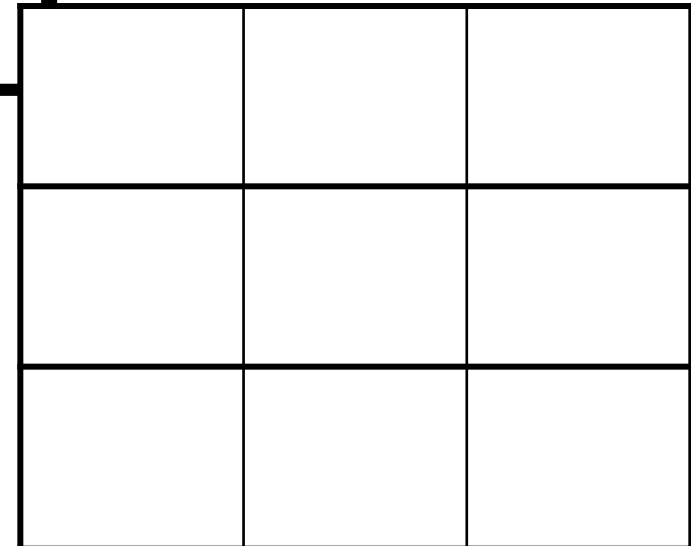
```
#include<stdio.h>
```



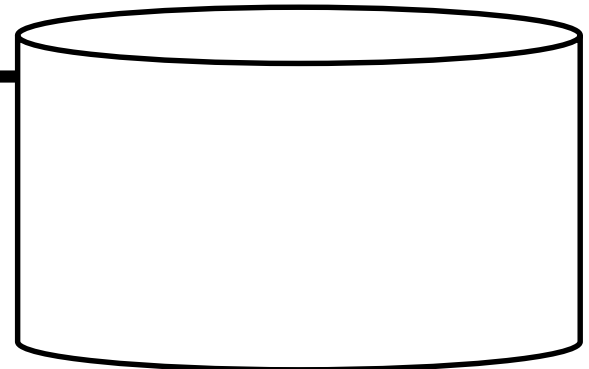
```
int main() {  
    float x;  
    int resposta;  
    printf("Digite um numero e direi se  
eh maior que 20: ");  
    scanf("%f", &x);  
    resposta = x > 20;  
    printf("Resposta: %d\n", resposta);  
    printf("Legenda: 0 significa NAO e  
1 significa SIM.\n");  
    return 0;  
}
```



Memória



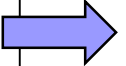
HD



Execução

```
#include<stdio.h>
```

```
int main() {
```



```
float x;
```

```
int resposta;
```

```
printf("Digite um numero e direi se  
eh maior que 20: ");
```

```
scanf("%f", &x);
```

```
resposta = x > 20;
```

```
printf("Resposta: %d\n", resposta);
```

```
printf("Legenda: 0 significa NAO e  
1 significa SIM.\n");
```

```
return 0;
```

```
}
```

Memória

X

HD

Execução

```
#include<stdio.h>
```

```
int main() {
```

```
    float x;
```

```
    int resposta;
```

```
    printf("Digite um numero e direi se  
    eh maior que 20: ");
```

```
    scanf("%f", &x);
```

```
    resposta = x > 20;
```

```
    printf("Resposta: %d\n", resposta);
```

```
    printf("Legenda: 0 significa NAO e  
    1 significa SIM.\n");
```

```
    return 0;
```

```
}
```

Memória

X

resposta

HD

Digite um numero e direi se
eh maior que 20:

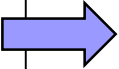
Execução

```
#include<stdio.h>
```

```
int main() {
```

```
    float x;
```

```
    int resposta;
```



```
    printf("Digite um numero e direi se  
eh maior que 20: ");
```

```
    scanf("%f", &x);
```

```
    resposta = x > 20;
```

```
    printf("Resposta: %d\n", resposta);
```

```
    printf("Legenda: 0 significa NAO e  
1 significa SIM.\n");
```

```
    return 0;
```

```
}
```

Memória

X

resposta

HD

Digite um numero e direi se eh maior que 20:

Execução

```
#include<stdio.h>
```

```
int main() {  
    float x;  
    int resposta;  
    printf("Digite um numero e direi se  
eh maior que 20: ");  
scanf("%f", &x);  
    resposta = x > 20;  
    printf("Resposta: %d\n", resposta);  
    printf("Legenda: 0 significa NAO e  
1 significa SIM.\n");  
    return 0;  
}
```

*A função scanf() espera uma digitação.
Vamos assumir que o usuário digita 38.*

Memória

X

resposta

HD

Digite um numero e direi se
eh maior que 20: 38

Execução

```
#include<stdio.h>
```

```
int main() {  
    float x;  
    int resposta;  
    printf("Digite um numero e direi se  
eh maior que 20: ");  
    scanf("%f", &x);  
    resposta = x > 20;  
    printf("Resposta: %d\n", resposta);  
    printf("Legenda: 0 significa NAO e  
1 significa SIM.\n");  
    return 0;  
}
```

A função scanf() espera um <ENTER>.

Memória

X

resposta

HD

Digite um numero e direi se
eh maior que 20: 38 <ENTER>

Execução

```
#include<stdio.h>
```

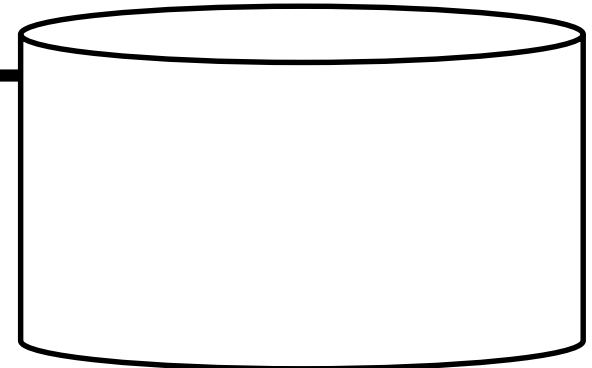
```
int main() {  
    float x;  
    int resposta;  
    printf("Digite um numero e direi se  
eh maior que 20: ");  
    scanf("%f", &x);  
    resposta = x > 20;  
    printf("Resposta: %d\n", resposta);  
    printf("Legenda: 0 significa NAO e  
1 significa SIM.\n");  
    return 0;  
}
```

*Ao clicar <ENTER>, 38 vai para a
memória.*

Memória

X	38	resposta	

HD



Digite um numero e direi se
eh maior que 20: 38 <ENTER>

Execução

```
#include<stdio.h>
```

```
int main() {  
    float x;  
    int resposta;  
    printf("Digite um numero e direi se  
eh maior que 20: ");  
    scanf("%f", &x);  
    resposta = x > 20;  
    printf("Resposta: %d\n", resposta);  
    printf("Legenda: 0 significa NAO e  
1 significa SIM.\n");  
    return 0;  
}
```

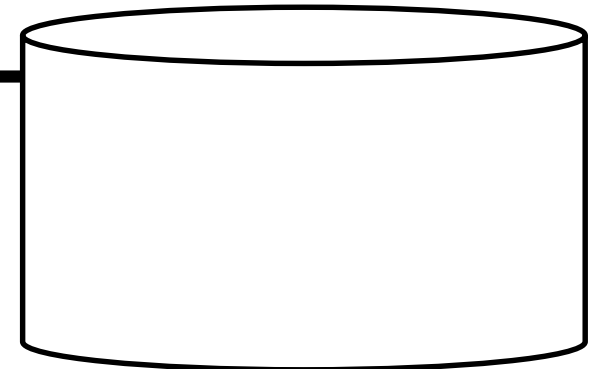
*Primeiro, o computador calcula, em uma
memória de rascunho, $x > 20$. Ou seja, ele
calcula:*

*$x > 20$
 $= 38 > 20$
 $= 1$ (verdadeiro)*

Memória

X	38	resposta	

HD



Digite um numero e direi se
eh maior que 20: 38 <ENTER>

Execução

```
#include<stdio.h>
```

```
int main() {
    float x;
    int resposta;
    printf("Digite um numero e direi se
eh maior que 20: ");
    scanf("%f", &x);
    resposta = x > 20;
    printf("Resposta: %d\n", resposta);
    printf("Legenda: 0 significa NAO e
1 significa SIM.\n");
    return 0;
}
```

*Primeiro, o computador calcula, em uma
memória de rascunho, $x > 20$. Ou seja, ele
calcula:*

$x > 20$

$= 38 > 20$

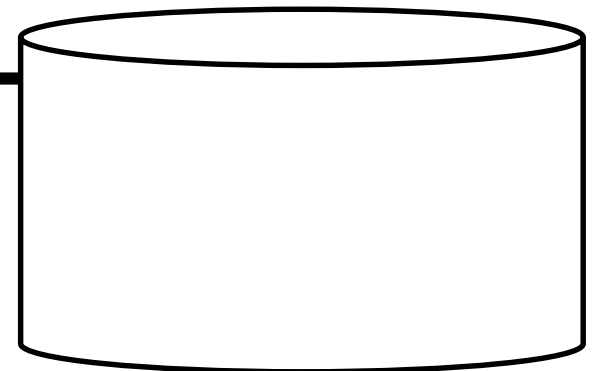
$= 1$ (verdadeiro)

Depois, o 1 vai para a memória

Memória

X	38	resposta	
		1	

HD



Execução

```
#include<stdio.h>
```

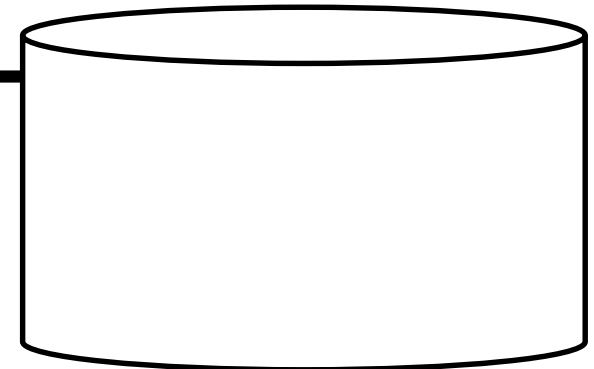
```
int main() {  
    float x;  
    int resposta;  
    printf("Digite um numero e direi se  
eh maior que 20: ");  
    scanf("%f", &x);  
    resposta = x > 20;  
    printf("Resposta: %d\n", resposta);  
    printf("Legenda: 0 significa NAO e  
1 significa SIM.\n");  
    return 0;  
}
```

Digite um numero e direi se
eh maior que 20: 38 <ENTER>
Resposta: 1

Memória

X	38	resposta	
		1	

HD



Execução

```
#include<stdio.h>
```

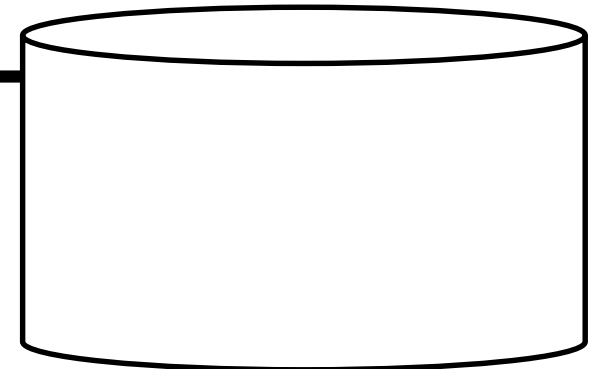
```
int main() {  
    float x;  
    int resposta;  
    printf("Digite um numero e direi se  
eh maior que 20: ");  
    scanf("%f", &x);  
    resposta = x > 20;  
    printf("Resposta: %d\n", resposta);  
    printf("Legenda: 0 significa NAO e  
1 significa SIM.\n");  
    return 0;  
}
```

Digite um numero e direi se
eh maior que 20: 38 <ENTER>
Resposta: 1
Legenda: 0 significa NÃO e 1
significa SIM.

Memória

X	38	resposta	
		1	

HD

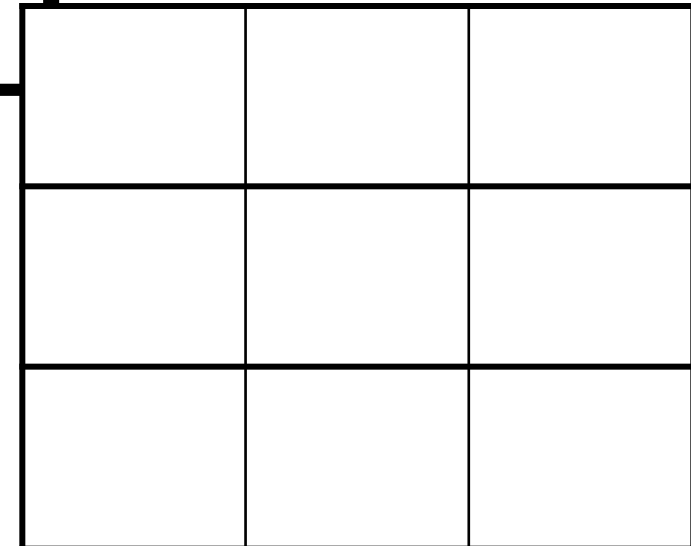


Execução

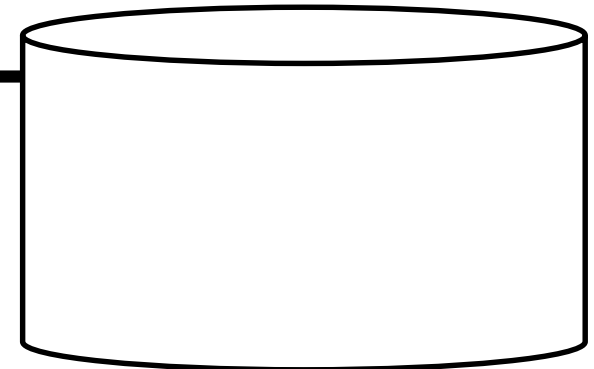
```
#include<stdio.h>
```

```
int main() {  
    float x;  
    int resposta;  
    printf("Digite um numero e direi se  
eh maior que 20: ");  
    scanf("%f", &x);  
    resposta = x > 20;  
    printf("Resposta: %d\n", resposta);  
    printf("Legenda: 0 significa NAO e  
1 significa SIM.\n");  
    return 0;  
}
```

Memória



HD



Operadores relacionais

■ Não confunda:

$x = 3$

- Grava o valor 3 em x
- Força que x seja igual a 3

$x == 3$

- Compara x com 3 e retorna 0 (Falso) ou 1 (Verdadeiro)
- O valor de x não muda

Operadores relacionais

■ O que vai aparecer no monitor?

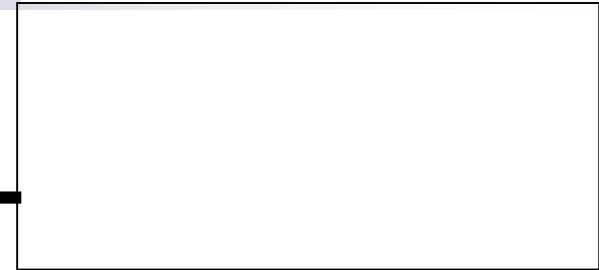
```
#include<stdio.h>
int main() {
    int x;
    int y;
    x = 8;
    y = 3;
    printf("%d \n", x == y);
    x = y;
    printf("%d \n", x == y);
    return 0;
}
```

Execução

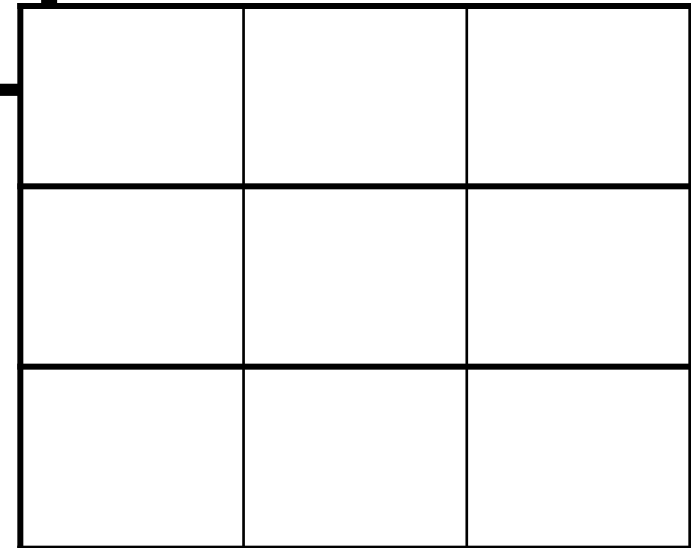
```
#include<stdio.h>
```



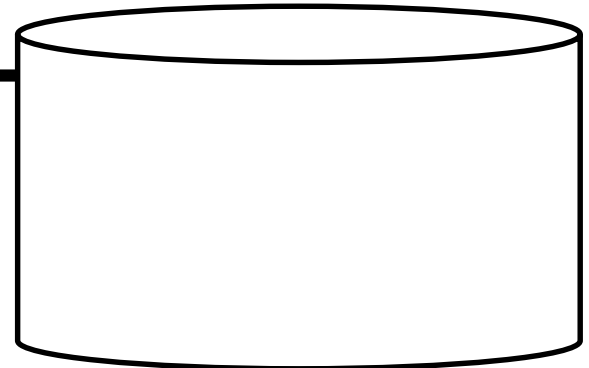
```
int main() {  
    int x;  
    int y;  
    x = 8;  
    y = 3;  
    printf("%d \n", x == y);  
    x = y;  
    printf("%d \n", x == y);  
    return 0;  
}
```



Memória



HD



Execução

```
#include<stdio.h>
```

```
int main() {
```



```
int x;
```

```
int y;
```

```
x = 8;
```

```
y = 3;
```

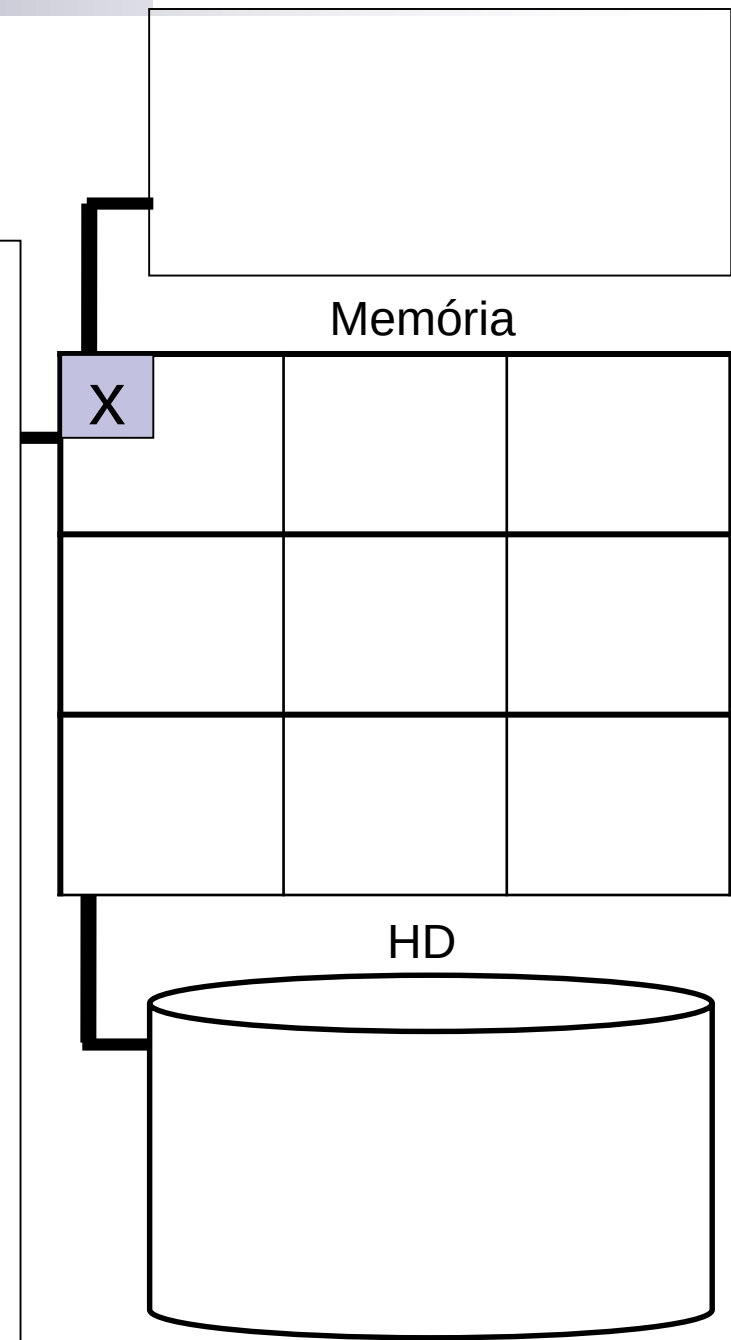
```
printf("%d \n", x == y);
```

```
x = y;
```

```
printf("%d \n", x == y);
```

```
return 0;
```

```
}
```



X

Memória

HD

Execução

```
#include<stdio.h>
```

```
int main() {
```

```
    int x;
```

```
    int y;
```

```
    x = 8;
```

```
    y = 3;
```

```
    printf("%d \n", x == y);
```

```
    x = y;
```

```
    printf("%d \n", x == y);
```

```
    return 0;
```

```
}
```

Memória

x

y

HD

Execução

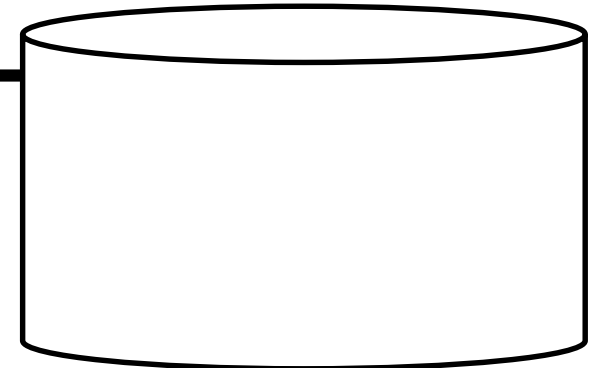
```
#include<stdio.h>
```

```
int main() {  
    int x;  
    int y;  
    x = 8;  
    y = 3;  
    printf("%d \n", x == y);  
    x = y;  
    printf("%d \n", x == y);  
    return 0;  
}
```

Memória

x	8	y	

HD



Execução

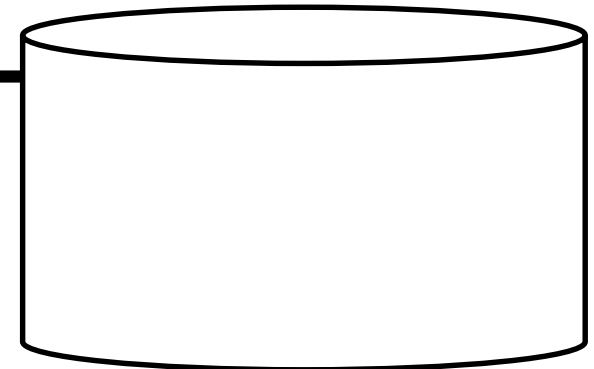
```
#include<stdio.h>
```

```
int main() {  
    int x;  
    int y;  
    x = 8;  
    y = 3;  
    printf("%d \n", x == y);  
    x = y;  
    printf("%d \n", x == y);  
    return 0;  
}
```

Memória

x	8	y	3	

HD



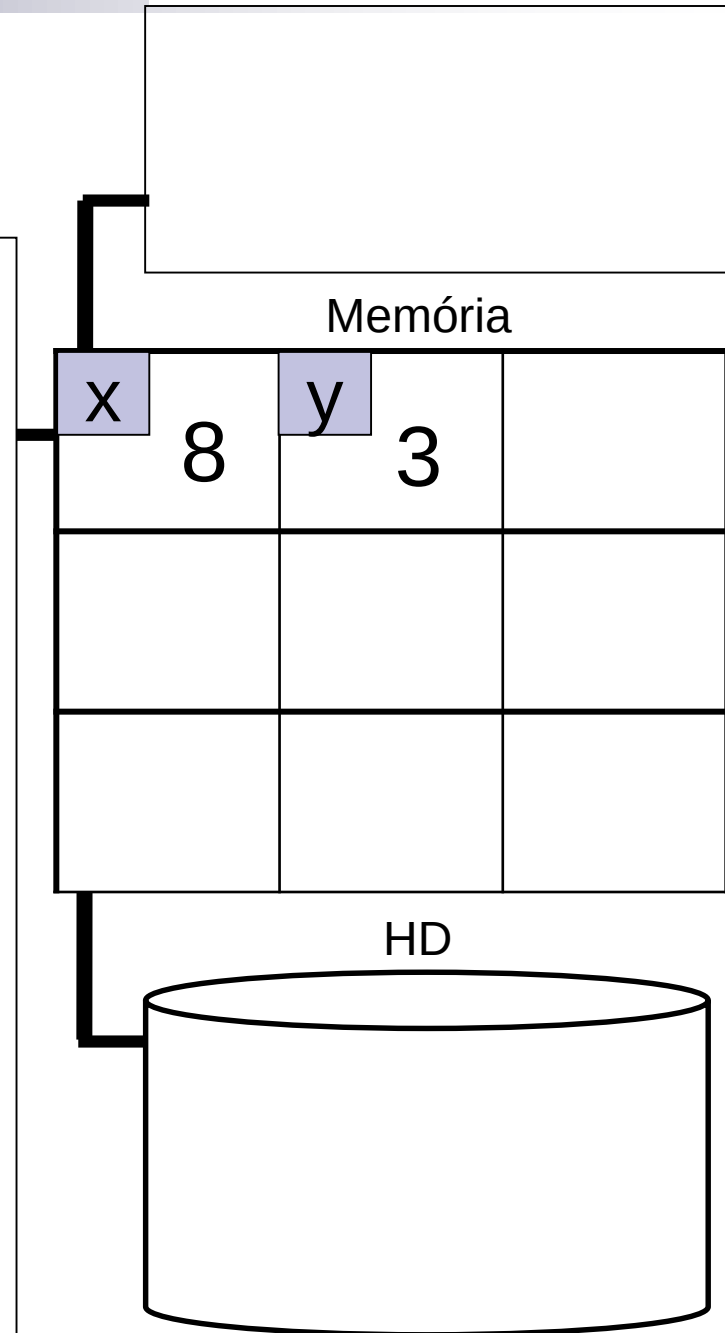
Execução

```
#include<stdio.h>
```

```
int main() {  
    int x;  
    int y;  
    x = 8;  
    y = 3;  
    printf("%d \n", x == y);  
    x = y;  
    printf("%d \n", x == y);  
    return 0;  
}
```

Primeiro, o computador calcula, em uma memória de rascunho, $x == y$. Ou seja, ele calcula:

*$x == y$
 $= 8 == 3$
 $= 0$ (falso)*



Execução

```
#include<stdio.h>
```

```
int main() {
    int x;
    int y;
    x = 8;
    y = 3;
    printf("%d \n", x == y);
    x = y;
    printf("%d \n", x == y);
    return 0;
}
```

Primeiro, o computador calcula, em uma memória de rascunho, $x == y$. Ou seja, ele calcula:

*$x == y$
 $= 8 == 3$
 $= 0$ (falso)*

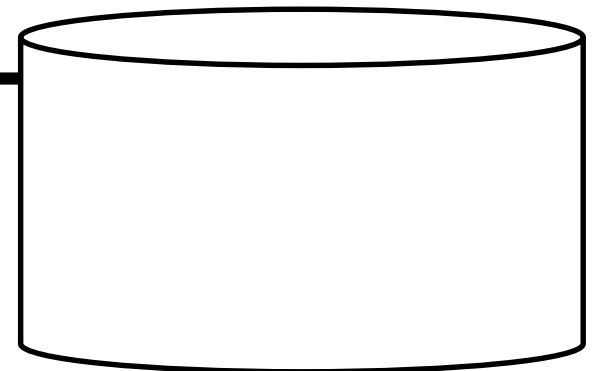
Depois, o 0 vai para o monitor.

0

Memória

x	8	y	3	

HD



Execução

```
#include<stdio.h>
```

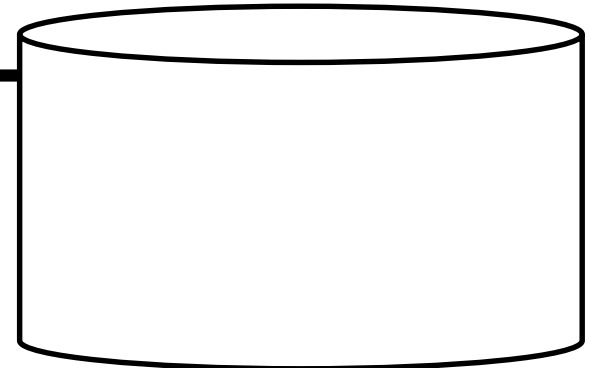
```
int main() {  
    int x;  
    int y;  
    x = 8;  
    y = 3;  
    printf("%d \n", x == y);  
    x = y;  
    printf("%d \n", x == y);  
    return 0;  
}
```

0

Memória

x	3	y	3	

HD



Execução

```
#include<stdio.h>
```

```
int main() {  
    int x;  
    int y;  
    x = 8;  
    y = 3;  
    printf("%d \n", x == y);  
    x = y;  
    printf("%d \n", x == y);  
    return 0;  
}
```

Primeiro, o computador calcula, em uma memória de rascunho, $x == y$. Ou seja, ele calcula:

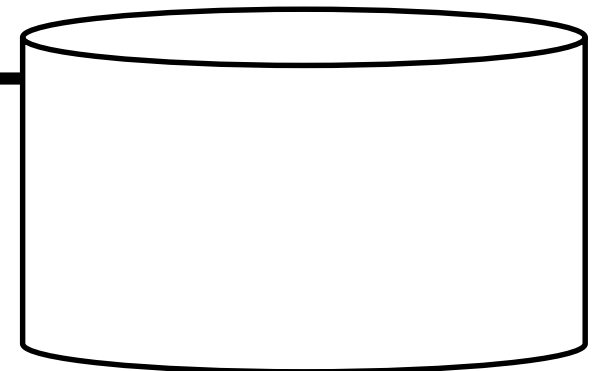
*$x == y$
 $= 3 == 3$
 $= 1$ (verdadeiro)*

0

Memória

x	3	y	3	

HD



Execução

```
#include<stdio.h>
```

```
int main() {  
    int x;  
    int y;  
    x = 8;  
    y = 3;  
    printf("%d \n", x == y);  
    x = y;  
    printf("%d \n", x == y);  
    return 0;  
}
```

Primeiro, o computador calcula, em uma memória de rascunho, $x == y$. Ou seja, ele calcula:

$x == y$

$= 3 == 3$

$= 1$ (verdadeiro)

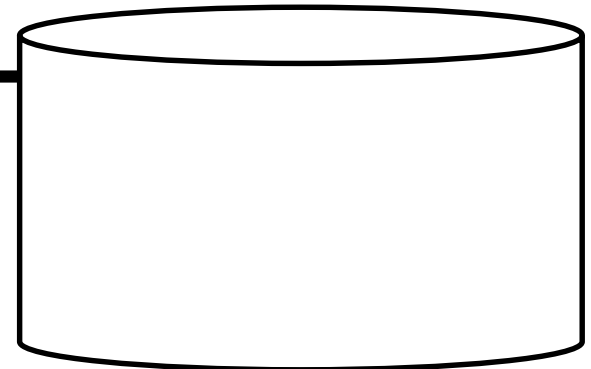
Depois, o 1 vai para o monitor.

0
1

Memória

x	3	y	3	

HD

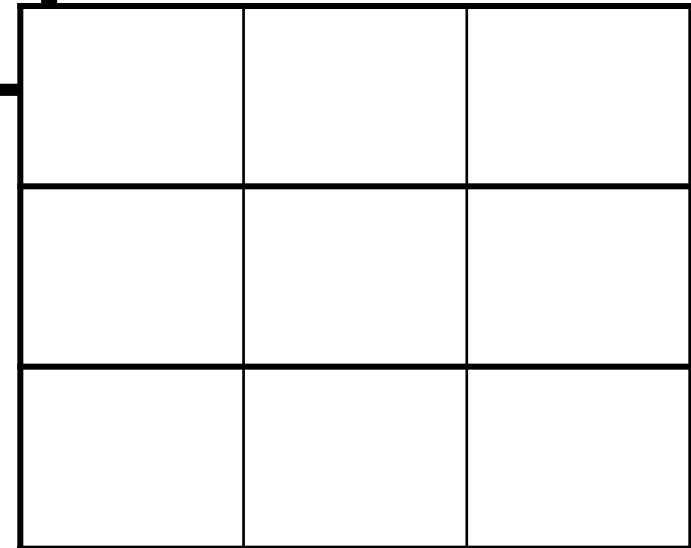


Execução

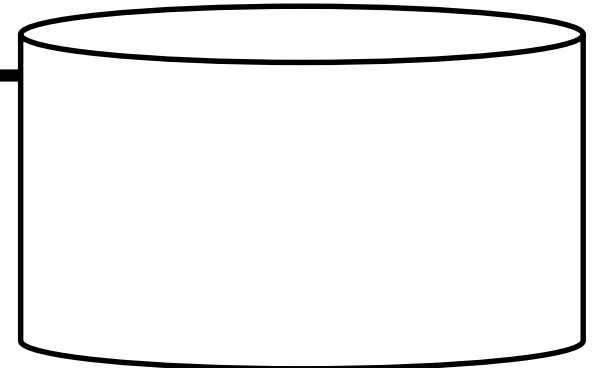
```
#include<stdio.h>
```

```
int main() {  
    int x;  
    int y;  
    x = 8;  
    y = 3;  
    printf("%d \n", x == y);  
    x = y;  
    printf("%d \n", x == y);  
    return 0;  
}
```

Memória



HD



Operadores Lógicos

E: &&

Ou: ||

Negação: !

a!

Neg.(!)	a
V	F
F	V

a && b

E(&&)	a	b
F	F	F
F	F	V
F	V	F
V	V	V

a || b

Ou ()	a	b
F	F	F
V	F	V
V	V	F
V	V	V

Expressões booleanas

```
#include<stdio.h>
int main() {
    int x,y,z;
    x = 8;
    y = 5;
    z = 3;
    printf("0 ou 1? %d\n", ((x > y) && (y > z)));
    printf("0 ou 1? %d\n", ((x > y) || (y > x)));
    printf("0 ou 1? %d\n", (!(x > y)));
    printf("0 ou 1? %d\n", ((x > 3) && (!(z < y) || (z == x))));
    printf("0 ou 1? %d\n", ((1 > 2) && ((1 > 3) == 0) && (0 == 0)));
    printf("0 ou 1? %d\n", ((1 > 2) && ((1 > 0) == 0) && (1 == 0)));
    return 0;
}
```

Exemplo

```
#include<stdio.h>
int main() {
    float nota1, nota2, nota3, media;
    int aprovado;
    printf("Entre as 3 notas: ");
    scanf("%f %f %f", &nota1,&nota2,&nota3);
    media = (nota1 + nota2 + nota3) / 3;
    aprovado = (media >= 7);
    printf("Passou (0=Nao,1=Sim)? %d ",aprovado);
    return 0;
}
```

Exemplo

```
#include <stdio.h>
```

```
int main() {  
  int x, y, z, b;  
  x = 3;  
  y = 4;  
  z = 8;  
  b = x > y;  
  printf("%d", b);  
  return 0;  
}
```



Substitua por:

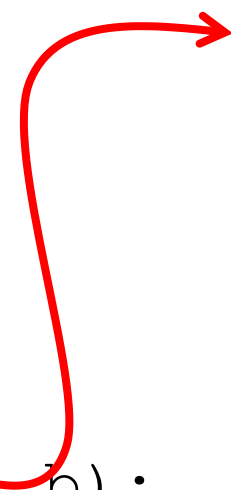
- $b = ((x*y) / z) > (y \% x);$
- $b = (z+y) == (x*y);$
- $b = (z \geq (2*y));$
- $b = (x+y) != (z-1);$

Qual o valor de b nestes casos?

Exemplo (&& - and)

```
#include <stdio.h>
```

```
int main() {  
    int x, y, z, b;  
    x = 3;  
    y = 4;  
    z = 8;  
    b = x > y;  
    printf("%d", b);  
    return 0;  
}
```



Substitua por:

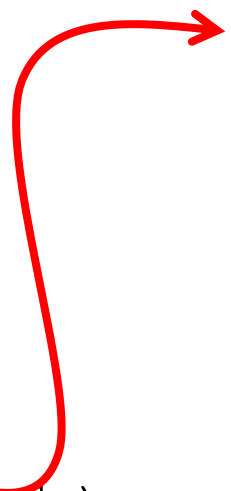
- $b = (x > y) \ \&\& \ (y > z);$
- $b = (x > 10) \ \&\& \ (z < 20);$
- $b = (z > 10) \ \&\& \ (y > 0);$
- $b = (x \leq 3) \ \&\& \ ((z / y) == 2);$

Qual o valor de b nestes casos?

Exemplo (|| - or)

```
#include <stdio.h>
```

```
int main() {  
    int x, y, z, b;  
    x = 3;  
    y = 4;  
    z = 8;  
    b = x > y;  
    printf("%d", b);  
    return 0;  
}
```



Substitua por:

- $b = (x > y) \parallel (y > z);$
- $b = (x > 10) \parallel (z < 20);$
- $b = (y > 0) \parallel (z > 10);$
- $b = (x \leq 3) \parallel ((z / y) == 2);$

Qual o valor de b nestes casos?

Exemplo (! - not)

```
#include <stdio.h>
```

```
int main() {  
    int x, y, z, b;  
    x = 3;  
    y = 4;  
    z = 8;  
    b = x > y;  
    printf("%d", b);  
    return 0;  
}
```

Substitua por:

- $b = !(x > y);$
- $b = !(z < 20);$
- $b = !((y > 0) \ \&\& \ (z > 10));$
- $b = !((x \leq 3) \ \&\& \ ((z / y) == 2));$

Qual o valor de b nestes casos?

Exercício

- Sejam 3 pontos A, B e C no plano, dados por suas coordenadas cartesianas (X_a, Y_a) , (X_b, Y_b) e (X_c, Y_c) . Fazer um programa em C para:
 - Ler, via teclado, as coordenadas dos 3 pontos;
 - Calcular as distâncias entre eles (D_{ab} , distância entre os pontos A e B, D_{ac} , distância entre os pontos A e C e D_{bc} , distância entre os pontos B e C);
 - Imprimir, no monitor, as distâncias calculadas.

$$d_{ab} = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}$$

Fórmula de D_{ab}

Exercício

- Todas as pessoas que se enquadram em um dos casos abaixo são obrigadas a declarar o Imposto de Renda:
 - ☐ Recebeu rendimentos superiores a **R\$ 15.764,28**
 - ☐ Recebeu heranças e doações cuja soma foi superior a **R\$ 40.000,00**
 - ☐ Tem patrimônio (imóveis, telefones, veículos, jóias e terra nua) de valor total superior a **R\$ 80.000,00**;
- Faça um programa que lê 4 valores reais: rendimentos, herancas, doacoes e patrimonio e imprime na tela 1 ou 0. 1 (Verdadeiro) significa que a pessoa tem que declarar imposto de renda. 0 (Falso) significa que ela não precisa declarar.